

Test UI Execution Observations

Paylocity Benefits Dashboard

Observations from ambiguities in the documentation during test execution

May 21, 2026 · Diego Lizarraga

Observations

001 — Pre-existing employee in shared test environment

An already-created employee exists in the environment:

ID	First Name	Last Name	Dependants	Salary	Gross	Benefits Cost / Net
bd306dec...	Steve	Rogers	1	52,000.00	2,000.00	57.69 / 1,942.31

Benefit calculations are correct: $(1000 + 1 \times 500) / 26 = 57.69$, net = 1942.31. Confirms the formula is working as expected.

Automation implication

Tests must never assert the table is empty at run start. Always locate employees by name, never by row index.

002 — JavaScript source code publicly accessible

The complete un-minified employeeClient.js is publicly accessible in the browser Sources tab. It reveals all API endpoint paths, HTTP methods, request payload shapes, environment detection logic (Prod/Stage/Local), and the existence of a Stage environment.

This also confirmed the root cause of BUG-UI-002 (column swap).

Recommendation

In production, minify and bundle JavaScript assets. Consider removing environment detection strings.

003 — UI only sends three fields to the API and username is omitted

The POST request payload from the UI contains only: firstName, lastName, dependants. No username field exists in the Add Employee form. The API accepts this request even when the username is marked as required in the swagger schema. Documented as BUG-API-001.

The response preview correctly shows the full server-computed response. The request payload is what the UI sends and the preview is what the server returns.

004 — Salary field present in API response and employee table

The employee table shows a Salary column displaying 52,000.00. The business rules state \$2,000/paycheck × 26 = \$52,000/year, which matches.

Open question for product owner

Is salary always fixed at \$52,000 annually or can it vary per employee? If fixed, why is it a settable field in the API schema?

005 — Anti-CSRF token present on login form → positive security finding

The login form POST includes a `_RequestVerificationToken`. This is a .NET anti-CSRF token that prevents malicious sites from submitting forms impersonating authenticated users. This is a positive security finding worth noting alongside BUG-UI-007.

006 — Empty state message works correctly

When no employees exist the table correctly displays 'No employees found.' spanning all 9 columns. FLAG-DEL-02 (empty table state) passes. Correct behavior.

007 — Add and Edit share the same modal → automation focus

One modal (`#employeeModal`) is used for both Add and Edit. Both Add and Update buttons exist in the DOM at all times and JavaScript toggles their visibility.

Automation focus

When locating the Add and Update buttons in tests, always scope to visible elements only to avoid interacting with the hidden Update button.

008 — Float value in dependants field silently truncated to integer

When a float value such as 2.5 is entered in the dependants field and saved, the employee is created with the value truncated to 2.

The dependants field uses `type='text'` in the HTML, not `type='number'`, so the browser doesn't enforce numeric input. The value is sent as the string '2.5' and the API or server-side logic performs the truncation silently.

Assessment

The swagger marks dependants as integer format. Silent truncation lets me know the user entered 2.5 but the system saved 2 with no feedback. It is debatable whether this is a bug, since some might say truncating is reasonable, others would say the API should reject 2.5 with a 400.

I would recommend clarification with the product owner on intended behavior.

Related

The dependants field input type should ideally be `type='number'` with `step='1'` to prevent non-integer input at the UI level

009 — Special characters and SQL injection strings accepted as literal strings — positive security finding

During manual testing the following inputs were submitted in `firstName` and `lastName`:

- O'Brien-123 — accepted and stored as a literal string. Apostrophes and hyphens are legitimate in real names.
- ' OR '1'='1 — accepted and stored as a literal string. No 500 error, no data leakage. Injection handled safely.

- Diego1234*^2! — accepted and stored as a literal string. No 500 error.

Assessment

The API correctly sanitizes and stores these inputs as literal strings rather than executing them. Appears to use parameterized queries or an ORM that handles input sanitization at the data access layer. Positive security finding.

Note

While these inputs were accepted, they were stored with the actual injection string as the employee name which might be a data quality concern if the system allows any string to be saved or manipulated in a database.

The product owner should confirm whether character restrictions are intended.